Reactive Optimizers: a short summary

The post introduces **OPAL (OPtimizer with an Agent in the Loop)**, a training setup in which an AI agent watches a model's training signals and can rewrite the optimizer while a run is in progress. Conventional optimizers follow a recipe chosen before training; OPAL instead lets its agent decide what evidence to inspect and how to adjust the update rule for different groups of parameters. It uses two editable programs: one extracts diagnostics from training, and the other applies the agent's current update rule.

The agent can react to changes such as stalled loss, gradient spikes, or disagreement between consecutive gradients. In the reported examples it changed learning-rate schedules, moved selected weights between AdamW and Muon, adjusted clipping, and averaged weights to reduce overfitting. The authors report improvements over starting recipes on several language and image-generation tasks, and show better recovery from repeated data-corruption stretches in a CIFAR-10 experiment. The results suggest that a training recipe can be adapted to the run rather than fixed in advance; they do not establish that an agent will improve every task.

One limitation is reproducibility. The agent makes decisions during training, so small differences in observations or reasoning can lead to different optimization paths. The authors say that most experiments were run once and only a few were repeated three times. They also note that benefits are smaller when runs are short and the baseline recipe is already well tuned.

Possible use in federated learning: a server could monitor round-by-round training loss, validation performance, client update norms, and cosine similarities between client updates. If updates begin to disagree sharply or progress stalls, a reactive controller could cautiously adjust the server learning rate, clipping threshold, or client contribution weights, then observe whether the next rounds improve. This is a proposed extension, not an experiment in the post. In a real federated system, exposing per-client updates or diagnostics may create privacy risks, so the signals and any aggregation would need to fit the system's privacy design.

Source: Xingjian Bai, Jia-Hao Liang, Jayden Teoh, and Vincent Sitzmann, “Reactive Optimizers”, October 2026.